

Python Code

```
1  """
2  Testing.py - RealEstateModel post-registration test suite.
3  Features confirmed via dt._columns_types_mapping inspection.
4  """
5
6  import os
7  import logging
8  import pickle
9  import numpy as np
10 import pandas as pd
11 from azure.ai.ml import MLClient
12 from azure.identity import DefaultAzureCredential
13
14 logging.basicConfig(level=logging.INFO, format="%(asctime)s %(levelname)s %(message)s")
15 log = logging.getLogger(__name__)
16
17 MODEL_NAME = "RealEstateModel"
18 MODEL_VERSION = None
19
20 FEATURE_NAMES = [
21     'Duration', 'PPD_Category', 'Postcode_Value', 'Locality_Value',
22     'Postcode_Density', 'Local_Premium_Ratio', 'Transaction_Year',
23     'Month_Sin', 'Month_Cos', 'District', 'Street',
24 ]
25
26
27 def get_client_and_model(model_name, version=None):
28     log.info("Connecting to Azure ML workspace...")
29     credential = DefaultAzureCredential()
30     client = MLClient.from_config(credential=credential)
31     if version:
32         model = client.models.get(name=model_name, version=version)
33     else:
34         versions = list(client.models.list(name=model_name))
35         if not versions:
36             raise ValueError(f"No registered model: '{model_name}'")
37         model = sorted(versions, key=lambda m: int(m.version))[-1]
38     log.info(f"Resolved: {model.name} v{model.version}")
39     return client, model
40
41
42 def load_model(client, model):
43     local_dir = f"./downloaded_models/{model.name}_v{model.version}"
44     mlflow_root = None
45     if os.path.exists(local_dir):
46         for root, _, files in os.walk(local_dir):
47             if "MLmodel" in files:
48                 mlflow_root = root
49                 break
```

```

50     if mlflow_root is None:
51         log.info(f"Downloading to: {local_dir}")
52         os.makedirs(local_dir, exist_ok=True)
53         client.models.download(name=model.name, version=model.version,
download_path=local_dir)
54         for root, _, files in os.walk(local_dir):
55             if "MLmodel" in files:
56                 mlflow_root = root
57                 break
58         log.info(f"Using: {mlflow_root}")
59         with open(os.path.join(mlflow_root, "model.pkl"), "rb") as f:
60             pipeline = pickle.load(f)
61         log.info(f"Loaded: {type(pipeline).__name__}")
62         return pipeline
63
64
65 def build_test_dataframe():
66     import math
67     def msin(m): return math.sin(2 * math.pi * m / 12)
68     def mcos(m): return math.cos(2 * math.pi * m / 12)
69
70     data = {
71         'Duration':      ['F',      'L',      'F',      'L',      'F'   ],
72         'PPD_Category':  ['A',      'A',      'B',      'A',      'A'   ],
73         'Postcode_Value': [280000, 450000, 195000, 620000, 310000],
74         'Locality_Value': [265000, 430000, 185000, 590000, 295000],
75         'Postcode_Density': [42,      78,      25,      110,     55   ],
76         'Local_Premium_Ratio': [1.06, 1.05, 1.05, 1.05, 1.05 ],
77         'Transaction_Year': [2022, 2023, 2021, 2023, 2022 ],
78         'Month_Sin':      [msin(m) for m in [3, 6, 9, 1, 11]],
79         'Month_Cos':      [mcos(m) for m in [3, 6, 9, 1, 11]],
80         'District':      [14,      27,      8,      42,      19   ],
81         'Street':         [203,     891,     115,    1204,   447   ],
82     }
83     df = pd.DataFrame(data)[FEATURE_NAMES]
84     log.info(f"Test DataFrame: {df.shape[0]} rows x {df.shape[1]} cols")
85     return df
86
87
88 def run_inference(pipeline, df):
89     log.info("Running inference...")
90     raw_preds = pipeline.predict(df)
91
92     print("\n--- RAW PREDICTION DIAGNOSTICS ---")
93     print(f" raw predictions : {raw_preds}")
94     print(f" min / max          : {raw_preds.min():.4f} /
{raw_preds.max():.4f}")
95     print(f" mean              : {raw_preds.mean():.4f}")
96
97     # Detect output scale automatically
98     if raw_preds.min() > 5 and raw_preds.max() < 20:
99         # Looks like log-scale - exponentiate to get f
100         scale = "log"
101         price_preds = np.exp(raw_preds)

```

```

102     log.info("Detected log-scale output → applying exp()")
103     elif raw_preds.min() > 10000:
104         # Already in £
105         scale = "direct"
106         price_preds = raw_preds
107         log.info("Detected direct £ output")
108     else:
109         scale = "unknown"
110         price_preds = raw_preds
111         log.warning(f"Unexpected prediction scale – raw values shown as-is")
112
113     print(f"    scale detected    : {scale}")
114     print("-----\n")
115
116     results = df.copy()
117     results["Raw_Prediction"] = raw_preds
118     results["Price_GBP_Predicted"] = price_preds
119     return results, scale
120
121
122 def run_sanity_checks(results, scale):
123     prices = results["Price_GBP_Predicted"]
124     checks = {
125         "No NaN or Inf predictions": (~prices.isna() &
~np.isinf(prices)).all(),
126         "All prices > £0": (prices > 0).all(),
127         "Prices are positive and finite":
128             prices.between(1_000, 10_000_000).all(),
129         "Price spread exists": prices.std() > 0,
130         "High-value postcode > low-value postcode (row 3 > row 2)":
131             results.loc[3, "Price_GBP_Predicted"] > results.loc[1,
"Price_GBP_Predicted"],
132     }
133
134     passed = failed = 0
135     print("\n" + "="*60)
136     print("    SANITY CHECK RESULTS")
137     print("="*60)
138     for check, ok in checks.items():
139         print(f"    {'✅ PASS' if ok else '❌ FAIL'} {check}")
140         passed += ok; failed += not ok
141     print(f"\n    {passed} passed | {failed} failed")
142     print("="*60 + "\n")
143     return failed == 0
144
145
146 def print_predictions(results):
147     labels = [
148         "Suburban freehold",
149         "City leasehold    ",
150         "Rural freehold     ",
151         "Prime London        ",
152         "Commuter belt       ",
153     ]

```

```
154     print("\n" + "="*60)
155     print("  PREDICTIONS")
156     print("="*60)
157     for i, row in results.iterrows():
158         label = labels[i] if i < len(labels) else f"Row {i+1}"
159         price = row["Price_GBP_Predicted"]
160         raw    = row["Raw_Prediction"]
161         if np.isfinite(price):
162             print(f" {label} → f{int(price):>10,} (raw={raw:.4f})")
163         else:
164             print(f" {label} → ⚠ non-finite (raw={raw:.4f})")
165     print("="*60 + "\n")
166
167
168 def main():
169     print("\n" + "="*60)
170     print("  RealEstateModel – Post-Registration Test Suite")
171     print("="*60 + "\n")
172
173     client, model = get_client_and_model(MODEL_NAME, MODEL_VERSION)
174     log.info(f"✅ Registry lookup OK (v{model.version})")
175
176     pipeline = load_model(client, model)
177     log.info("✅ Model loaded OK")
178
179     df = build_test_dataframe()
180     results, scale = run_inference(pipeline, df)
181     log.info("✅ Inference OK")
182
183     print_predictions(results)
184     all_passed = run_sanity_checks(results, scale)
185     log.info("✅ All checks passed" if all_passed else "⚠ Some checks failed")
186
187     results.to_csv("model_test_results.csv", index=False)
188     log.info("Saved → model_test_results.csv")
189
190
191 if __name__ == "__main__":
192     main()
```